Orientation, the device spectrum, and how code runs

Course plan, device spectrum, and code execution

Dinu-Ștefan Rusu MEC

What you should leave with

Course plan

The 14 lectures, the 7 labs, the 7 project sessions, and how the 10 points are earned.

Device spectrum

Where a phone, a single-board computer and a microcontroller sit, and how their budgets differ.

Compilation models

Compilers, interpreters, JIT and AOT, and why Dart is built both ways.

Memory on a phone

What a garbage collector does for you, and what it costs inside a frame.

1

Course organization

Lectures, labs, project, grading

Contents

		Page
1	**Course organization**	3
2	From the cloud to a microcontroller	13
3	Embedded systems and the phone as hub	19
4	Code execution	22

Instructors

Andrei Vasilățeanu

Dinu-Ștefan Rusu
dinu_stefan.rusu@upb.ro

Microsoft Teams: the course channel.
Everything else: rusudinu.com

Course repository
github.com/rusudinu/presentations

50+
apps shipped

500k+
installs across them

Course scope

Under the app

How code executes, how the phone spends energy, how a microcontroller is programmed.

Around the app

Data that survives without a network, servers over RPC, devices over MQTT and BLE.

Not the app itself

Widgets, layout and screens belong to the sibling course, ADMD.

The stack for the whole semester

Flutter and Dart on the phone. Go on the server. TinyGo on the microcontroller. MQTT between them.

The 14 lectures

Wk	Topic
1	Orientation, devices, how code runs
2	Concurrency: event loop and isolates
3	Embedded fundamentals and TinyGo
4	Device networking: MQTT, CoAP, TLS
5	Bluetooth Low Energy
6	Phone sensors and hardware APIs
7	Background execution and power

Wk	Topic
8	Offline-first: local data and sync
9	Offline-first: conflicts and CRDTs
10	Backends: serverless, VPS, and Go
11	gRPC, protocol buffers, HTTP/2 and 3
12	Platform channels, FFI, and Kotlin MP
13	Rendering and optimization
14	Compilation, energy, revision

Every deck goes to Moodle on the day it is presented, and the whole set stays in the course repository.

Labs and project sessions

Week	Deck	Lab
2	lab01	Toolchain, board, broker, project kickoff
4	lab02	A sensor reading to the phone over MQTT
6	lab03	BLE: a peripheral and a Flutter central
8	lab04	Phone sensors and a background task
10	lab05	Offline-first: outbox, sync, one conflict rule
12	lab06	gRPC: a Go server and a Dart client
14	lab07	Lab test

Lecture weekly, lab in even weeks, project in odd weeks. Every session is 2 hours.

Grading

40 p
final exam, in the exam session

30 p
laboratory and assignments

15 p
lab test, in week 14

15 p
team project, teams of at most 3

50
points needed to pass

Sixty of the hundred points come from work done during the semester.

Concept presentations

- Teams of at most 3 pick a topic from the shared sheet. Topics are taken in the order they are claimed.
- Prepare about 20 minutes of slides, with a demo if the topic supports one, inside the same 20 minutes.
- Every member presents a part, from your own laptop. Materials go to Moodle.

Speaking time

20 minutes divided by team size. A team of 2 speaks about 10 minutes each, a team of 3 about 7 minutes each.

Claim a topic in the first three weeks.

Project requirements

1. A device component

A TinyGo program on a board, or in the Wokwi simulator, reading a sensor.

2. Device to phone

The app shows the reading live, over MQTT or over BLE.

3. Offline-first

The app works with no network and syncs later, with a conflict rule you state.

4. gRPC or FFI

A Go server with a generated Dart client, or native code called from Dart.

5. A measured claim

One number you measured: energy, latency, payload size or frame time, with the method.

Entry conditions

State in BLoC, real authentication, one WebSocket screen. Your ADMD app has these.

Project milestones

Week	What must be true at the end of the session
1	Team formed, at most 3 people, and the idea approved
3	The device streams a reading to a serial console
5	The app receives that reading over MQTT or BLE
7	Offline mode works, and the queued writes sync
9	gRPC or FFI is integrated and called from the app
11	The measurement is done, and the app is polished
13	Demo and defense

Teams of at most 3. Approved in Lab 1, defended in the last project session.

Prerequisites

Prerequisite: ADMD, or equivalent Flutter and Dart knowledge.
You should be able to build a screen, hold state in BLoC, call an HTTP API and sign a user in.

If you took Introduction to SAP instead, work through the ADMD Lecture 2, 3, 5 and 6 decks before week 3. Lecture 2 here assumes async and await are familiar.

Before week 3

The Flutter SDK installed, an emulator or a device that runs your ADMD app, and a GitHub account you can push from.

This course does not reteach widgets, layout or Dart syntax. When a topic belongs to ADMD, the slide names the ADMD lecture and moves on.

2 From the cloud to a microcontroller

Contents

		Page
1	Course organization	3
2	**From the cloud to a microcontroller**	13
3	Embedded systems and the phone as hub	19
4	Code execution	22

The device spectrum

	Cloud	Phone	Board	MCU
RAM	hundreds of GB	8–16 GB	1–8 GB	about 0.5 MB
Storage	many terabytes	128–512 GB	SD card	about 4 MB flash
Power	grid, kilowatts	about 5 W, battery	about 5 W	milliwatts
System	Linux	Android or iOS	Linux	an RTOS, or none
Code	Go, Java	Dart, Kotlin, Swift	Go, Python	TinyGo, C

This course lives on the right half of the table. Board means a single-board computer, MCU a microcontroller.

Mobile and embedded constraints

Battery is a resource

Every cycle and every radio wake-up costs energy your code cannot replenish.

Thermal limits

Sustained load throttles the chip, so extra CPU time is not always available.

Intermittent connectivity

Offline is a normal state, not an error. Lectures 8 and 9 build on that.

No control of deployment

Store review, staged rollouts, and users who never update the app.

Sensors and radios

GPS, accelerometer, camera, BLE. Capabilities a desktop does not have.

ARM almost everywhere

From flagship phones down to many boards, the instruction set is ARM.

Device resources

CPU
Compute, shared with every other scheduled process.

RAM
Working memory. The OS reclaims it by killing you.

Storage
Flash, far slower than RAM, shared with photos.

GPU
Graphics, and on-device machine learning.

Battery
Mobile only. A budget your code never refills.

Radios
Cellular, Wi-Fi and BLE, each with its own cost.

On a desktop the first four are effectively elastic. On a phone all six are hard budgets.

Resource allocation: loading a model

1. The app asks the operating system to load the model weights.
2. Enough free GPU memory: they land there and inference runs fast.
3. Otherwise, enough free RAM: they land there and inference runs, slower.
4. Neither: the load fails, and the app falls back to a server.

Unified memory

Phones and Apple laptops share one memory pool between CPU and GPU, so the effective graphics memory is larger than on a desktop with a separate card.

On-device machine learning is a resource problem first. A model that runs on your laptop may not load at all on a mid-range phone.

Key figures

8–16 GB
RAM in a 2026 flagship phone

520 KB
SRAM on an ESP32

5 W
peak draw of a phone chip, on a battery

4 MB
flash on a common ESP32 module

8.3 ms
one frame at 120 Hz

16.7 ms
one frame at 60 Hz, for comparison

One frame budget equals one divided by the refresh rate. Most phones sold in 2026 run at 90–120 Hz, so the familiar 16 ms describes a 60 Hz screen.

3

Embedded systems and the phone as hub

Kilobytes of RAM, milliwatts of power, no operating system

Contents Page

1 Course organization 3
2 From the cloud to a microcontroller 13
3 Embedded systems and the phone as hub 19
4 Code execution 22

Embedded constraints

- Kilobytes of RAM instead of gigabytes, milliwatt power budgets, and often no operating system at all.
- No process to kill and no swap. Allocate more than the chip has and the program stops.
- The program is one loop: wake, measure, send, sleep. Lecture 3 writes that loop.

Boards for this course

TinyGo supports the ESP32-C3 and ESP32-S3, with Wi-Fi and Bluetooth through the espradio package, and the Raspberry Pi Pico. The original ESP32 gets Wi-Fi, but no Bluetooth.

Buy an ESP32-C3, an ESP32-S3 or a Raspberry Pi Pico. If no board arrives in time, the Wokwi simulator is accepted, and Lab 1 names the exact simulator board.

The phone as hub

Sensor and microcontroller

A TinyGo program reads a sensor and publishes it. Milliwatts, no screen, no user.

Your Flutter app

Receives over MQTT or BLE, stores locally, shows it live, survives a lost network.

Cloud

Storage, dashboards and heavier processing, reached when a connection is worth using.

Where the semester goes

Lectures 3 and 4 build the left box. Lectures 5 to 9 build the middle box. Lectures 10 and 11 build the right box.

The phone is the only device in the chain with a screen, a battery worth charging and a network stack you trust.

4

Code execution

Compilers, interpreters, JIT, AOT, and garbage collection

Contents Page

1 Course organization 3
2 From the cloud to a microcontroller 13
3 Embedded systems and the phone as hub 19
4 Code execution 22

Compilation stages

1. Source text

Characters in a file. The CPU can execute none of it, so something must translate it.

2. An intermediate form

Tokens, a syntax tree, then bytecode or a compiler representation. Type checking happens here.

3. Machine instructions

ARM or x86 opcodes for one CPU and one operating system.

4. Execution

The instructions run, over a runtime that handles memory, threads and system calls.

Toolchains differ in when step 3 happens. Before you ship, on the device at start-up, or once a function turns out to be hot.

Compiler and interpreter

- A compiler translates the whole program ahead of time, for one CPU and one OS. You ship the result, and the translation is paid once, by you.
- An interpreter reads your code and carries out its instructions on every run. You ship the source, and the user's device pays.
- Almost nothing real is purely one or the other.

One language, run two ways

Python has an interpreter and a JIT implementation. Java and Kotlin interpret bytecode first, then compile the hot methods. Browser engines do both.

Compiled and interpreted describe a toolchain, not a language. The same source can be run either way, and for Dart it is.

The compilation spectrum

	AOT	JIT in a VM	Bytecode VM	Interpreter
Translated	before you ship	while running	at load time	line by line
You ship	machine code	source, a VM	bytecode, a VM	source, a runtime
Start-up	fastest	needs warm-up	medium	starts instantly
Peak speed	high and steady	can beat AOT	medium	lowest
Examples	Go, Rust, Swift	JVM, V8	CPython	shell scripts

Most runtimes mix these approaches. Android's ART mixes all three: ahead-of-time compilation, a JIT, and profiles collected on the device.

Compiled versus interpreted performance

- There is a gap, and it depends on the workload. A tight numeric loop shows the largest difference. Code dominated by input and output shows almost none.
- A JIT can beat an ahead-of-time compiler, because it sees the actual types and hot branches at run time.
- AOT wins where a phone needs it: cold start, memory footprint, predictable latency.

⚠ Not a fact

”Compiled languages are about a hundred times faster than interpreted ones.” There is no single multiplier. Ask instead: faster at what, measured how, on which device.

Benchmark the workload you actually have. Inside a frame, the budget goes to layout, rasterization and storage long before language dispatch.

Dart JIT and AOT compilation

Debug: JIT

```
flutter run
```

The VM compiles code as it runs. Hot reload swaps new code into the live isolate. Assertions on, DevTools attached. Slower, and a much larger binary.

Release: AOT

```
flutter build apk --release
```

Compiled ahead of time to native ARM machine code. No compiler ships inside the app, only a small runtime. Fast cold start, smaller footprint. No hot reload, because there is nothing to reload into.

Never judge performance from a debug build. Measure in profile or release mode. A debug build can be several times slower, by design.

Garbage collection

- Dart is garbage collected. You never free memory. The runtime reclaims objects that nothing can reach any more.
- Most objects die young. A rebuild allocates thousands of short-lived widget objects and drops them a frame later.
- So the heap is split: a young space collected constantly and cheaply, an old space collected rarely and mostly concurrently.

Three parts

Young space: almost everything dies here. **Scavenge:** copy the few survivors, drop the rest. **Old space:** collected rarely, alongside your code.

Why it matters on a phone. A pause longer than one frame budget is a visible dropped frame, and every collection costs cycles and therefore battery.

Summary

1. **One spectrum, cloud to microcontroller.** This course lives on the constrained half, where the machine runs on a battery.
2. **Compiled and interpreted describe a toolchain.** Dart sits at two points on that spectrum: JIT while you develop, AOT for what you ship.
3. **Garbage collection costs frame time.** A collection pause inside a frame budget is a dropped frame the user can see.

Further reading

tinygo.org · dart.dev on the Dart runtime · docs.flutter.dev/perf · mqtt.org

Before Lab 1

Form your team

At most 3 people. Names and idea on the team sheet before the first project session.

Choose a board

Order an ESP32-C3, an ESP32-S3 or a Raspberry Pi Pico, or commit to the Wokwi simulator.

Bring a working app

Your ADMD app, running, with state in BLoC, authentication and one WebSocket screen.

Also this week

Claim a concept-presentation topic, and read the five project requirements again before you commit to an idea.

The team and the board are the two decisions that block everything else. Both are due in Lab 1, in week 2. A board ordered in week 4 will not arrive in time for Lab 2.

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel · rusudinu.com